int a[] arr = new int[5] System.in)

CSE-207

1. Create Database DB-name;
2. Create table Tbl-name(Col-name1 datatype(size).....)

Student:

create table Student(Name varchar(35), St_ID int(2), NID int(3), Intake int(2), Section int(1), CGPA float(3,2))

Value/Data insertion in table:-

insert into Table-name values(value 1, value 2, ..., value n)

insert into Student values ("Ashraful", 1, 102, 44, 1, 3.80);
" " " " ("Tonny", 2, 101, 44, 1, 3.50);
" " " " ("Zakir", 33, 103, 35, 2, 3.98);
" " " " ("Urmi", 6, 104, 35, 1, 3.96)

CamScanner

Sub: Sat Sun Mon Tue Wed Thu Fri

Date: / /

" " " " ("Tonny", 23, 105, 11, 1, 3.50)

Faculty:

create table Faculty (Name varchar(35), Short_Code varchar(5), Course_Code varchar(10), Conduct_Semester varchar(15))

insert into Faculty values("Farha Akter Munmun", "FAM", "CSE 319", "Summer 2019")

Course:

Create table Course (Course_Title varchar(20), ~~Course varchar~~ Course_Code varchar(10))

Sub: _____

Sat Sun Mon Tue Wed Thu Fri

Date: / /

3(a)

Update Faculty set Short_Code = "NAF"
where Name = "Nadia Afrin Ritu";

b) Update Course set Course_Code = "CSE 317"
where Course_Title = "System Analysis Design";

c) Update Faculty set Conduct_Semester = "Fall 2020"
where Name = "Nadia Afrin Ritu" AND
~~Nam~~ Course_Code = "CSE 351";

d) ADD column

Alter table ~~ta~~ tbl-name ADD column col-name datatype(size)

Alter table Student ADD column Age int(2);

i) ADD column after a recognize column

At Alter table Course ADD column offered-semester varchar(10)
AFTER Course_Title.

n. Delete those Students from Student table whose CGPA is less than 3.40.

Delete from tbl_name where condition to identify the targeted row

Delete from Student where CGPA < 3.40;

Column Delete :-

Alter table tbl_name Drop Column col-name ;

"NID" Delete from Student's table:

Alter table Student Drop Column NID;

Sub: _____ Date: / /

k) Select name from Student
where Age > 22 AND CGPA > 3.50;

i) Select St_ID from Student
where ~~int~~ Intake = 44 ~~AND~~ OR CGPA < 3.40

m) Select St_Id from Student
~~where CGPA = 3.30 OR CGPA~~
where CGPA IN (3.30, 3.50, 3.96);

If those are not in those CGPA then,

* Select St_ID from Student
where CGPA NOT IN (3.30, 3.50, 3.96)

Data Defination Language (DDL)

DDL

Create
Alter → ADD
Alter → DROP

DML

Insert
Delete
Update

Scheme defination :- Student (Name, St_ID, NID, Intake, Section, CGPA)
If Scheme defination is ~~not~~ unchanged, then it is DML, else DDL

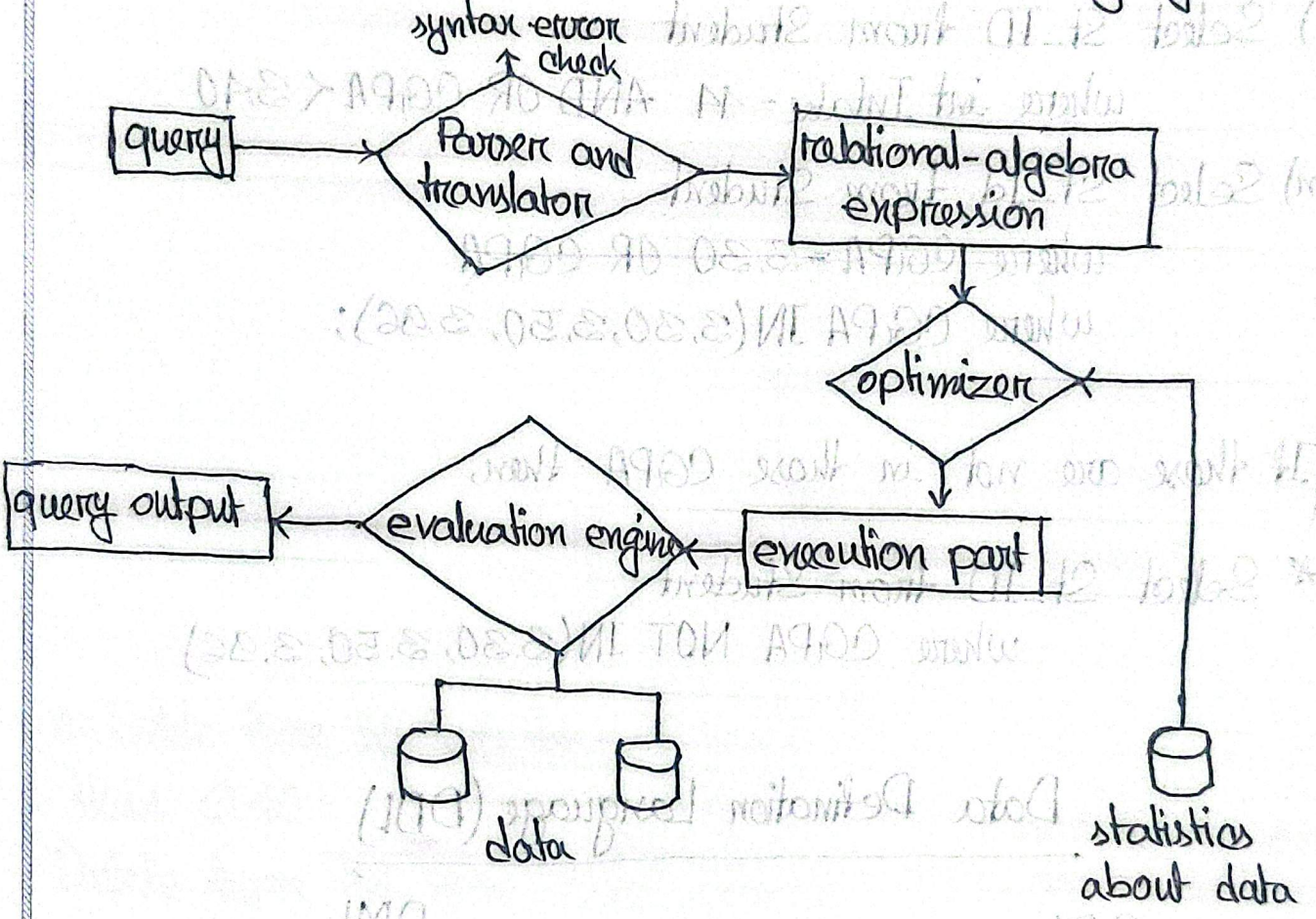
(Schema defination != changed) ? DML : DDL;

Sub: _____

Sat Sun Mon Tue Wed Thu Fri

Date: / /

* Query processing is the process of translating/converting high level SQL into low level machine language.



* Database Users

4 types of ~~user~~ users:

(i) Naive users

(ii) Application programmers

(iii) Sophis

(iv) Specialized ~~use~~ users

Sub: _____

Sat Sun Mon Tue Wed Thu Fri

Date: / /

* Database Administrator

** Chapter-2: Intro to Relational Model

attribute → column

entity → row / tuple

* Relational Algebra:

(i) Select: σ

(ii) project: π

(iii) union: $\cup$

(iv) Intersection: $\cap$

(v) Set difference: $-$ or $\setminus$

(vi) Cartesian product: α x

(vii) Rename: ρ

* Select: Selected Row and all columns.

q: Select those tuples of the instructor relation where the instructor is in the "Physics" department.

Query:- Select * from Instructor where dept_name = "Physics"

R.A.: $\sigma_{condition}(tbl\text{-}name)$

$$\sigma_{dept_name = "Physics"}(Instructor)$$

CamScanner

Sub: _____ Sat Sun Mon Tue Wed Thu Fri

Date: / /

q: Find the instructor in Physics with a salary greater 90,000.

SQL: Select * from Instructor where dept_name = "Physics" AND Salary > 90000;

R.A.: $\sigma_{dept_name = "Physics" \wedge Salary > 90000}(Instructor)$

* Project: Selected columns and all rows.

q: eliminate the dept_name attribute of instructor OR Select Show the Id, Name and Salary of each Instructor.

SQL: Select Id, name, Salary from Instructor.

R.A.: $\Pi_{Id, Name, Salary}(Instructor)$

* Composition of Select and Project Operation (Select + Project)

SQL: Select Name from Instructor where dept_name = "Physics";

R.A.: $\Pi_{Name}(\sigma_{dept_name = "Physics"}(Instructor))$.

Sub: _____ Sat Sun Mon Tue Wed Thu Fri

Date: / /

9: Find the name and Id of those students whose CGPA is greater than 3.50 and Age is greater that than 22.

SQL: ~~Select from Instructor where CGPA > 3.50 AND Age > 22;~~

Select Name, Id from Student where CGPA > 3.50 AND Age > 22

R.A.: $\pi_{Name, Id}(\sigma_{CGPA > 3.50 \wedge Age > 22}(Student))$

~~*i) π_{Name}(~~

~~j) $\pi_{Name}(\sigma_{CGPA > 3.50}$~~

* i) Select Course_name from Course;

R.A.: $\pi_{Course_name}(Course)$

h) Select Name from Student where CGPA > 3.50;

R.A: $\pi_{Name}(\sigma_{CGPA > 3.50}(Student))$

k) Select Name from Student where Age > 22 AND CGPA > 3.50;

R.A.: $\pi_{Name}(\sigma_{Age > 22 \wedge CGPA > 3.50}(Student))$

Sub: _____ Sat Sun Mon Tue Wed Thu Fri

Date: / /

1) Select DISTINCT St_Id from Student where Intake = 44 OR CGPA < 3.40;

R.A.: $\pi_{St_Id}(\sigma_{Intake = 44 \vee CGPA < 3.40}(Student))$

* Union:

Actor

Name	Gender
AJ	M
Jk	M
HA	M
AB	F
PN	F

Singer

Name	Gender	City
TK	M	Dhaka
HA	M	"
TF	F	"

* Not possible to Union

* It only ask for name or gender, then the union is possible.

Condition for performing:

* Union Operation:

i) Two table must have the same number of attribute.

(ii) The ~~ab~~ attribute domains must be compatible.

SQL for ~~Actor and~~ union of Actor and Singer (without city)

(Select Name, Gender from Actor) Union (Select Name, Gender from Singer)

R.A.: $\pi_{Name, Gender}(Actor) \cup \pi_{Name, Gender}(Singer)$

CamScanner

Sub: _____

Sat Sun Mon Tue Wed Thu Fri

Date: / /

*Intersection:

From Actor and Singer (without city) table.

⇒

Name	Gender
HA	M

SQL: (Select Name, Gender from Actor) Intersection (Select Name Gender from Singer);

R.A.: $\pi_{Name, Gender}(Actor) \cap \pi_{Name, Gender}(Singer)$

* Set difference:

From Actor and Singer (without city) table:

⇒

Name	Gender
AJ	M
Jk	M
AB	F
PN	F

SQL: (Select Name, Gender from Actor) Minus (Select Name, Gender from Singer);

R.A.: $\pi_{Name, Gender}(Actor) - \pi_{Name, Gender}(Singer)$

CamScanner

Sub: _____

Sat Sun Mon Tue Wed Thu Fri

Date: / /

* Cartesian Product:

Student

Id	Name
1	A
2	B
3	C

×

Home

Id	House no
1	A-11
2	B-12
5	E-15

⇒

Student.Id	Name	Home.Id	House no.
1	A	1	A-11
1	A	2	B-12
1	A	5	E-15
2	B	1	A-11
2	B	2	B-12
2	B	5	E-15
3	C	1	A-11
3	C	2	B-12
3	C	5	E-15

R.A.: $\Pi_{Name}(\sigma_{House_no = "B-12"} \wedge Student.Id = Home.Id (Student \times Home))$

* Cartesian Product:

~~Stu~~

Q1. Find the name of the student who lives in 'B-12'.

SQL: ~~Where~~ Select Name from Student, Home where House_no="B-12" ~~where~~ AND Student.Id = Home.Id ;

Select Student.Id, Name, Home.Id, House_no from Student, Home;

Sub: _____

Sat Sun Mon Tue Wed Thu Fri

Date: / /

* Natural Join: (⋈)

Id	Name	House no.
1	A	A-11
2	B	B-12

SQL: Select Name from Student Natural Join Home where House_no = "B-12";

~~R.A.: $\pi_{Name}(\sigma_{House_no = "B-12"}(Student$~~

R.A.: $\pi_{Name}(\sigma_{House_no = "B-12"}(Student \bowtie Home))$

Loan

L-no	b-name	amount
L-11	RH	900
L-14	DT	1500
L-15	PR	1500
L-16	PR	1300
L-17	DT	1000

Borrower

c_name	L-no
Adams	L-16
Smith	L-11
Jones	L-93
Jackson	L-14

Q1. Perform Natural Join (O/P + SQL + RA)

Q2. Find the name of those customers whose loan amount is greater than 1000. (SQL + RA)

CamScanner

Sub: _____

Date: / /

Q1.

L_no	b_name	amount	c_name
L-11	RH	900	Smith
L-14	DT	1500	Jackson
L-16	PR	1300	Adams

SQL: Select L_no, b_name, amount, c_name from Loan Natural Join Borrower; (or *)

R.A.: $\Pi_{L_no, b_name, amount, c_name}$ (Loan ⋈ Borrower)

Q2. SQL; Select c_name from Loan, Borrower (Natural Join) where amount > 1000;

R.A.: $\Pi_{c_name}(\sigma_{amount > 1000}$ (Loan ⋈ Borrower))

* Left Outer Join (⟕):

L_no	b_name	amount	c_name
L-11	RH	900	Smith
L-14	DT	1500	Jackson
L-15	PR	1500	NULL
L-16	PR	1300	Adams
L-17	DT	1000	NULL

SQL: Select * from Loan Natural Left Outer Join Borrower;

R.A.: $\Pi_{L_no, b_name, amount, c_name}$ (Loan ⟕ Borrower)

Sub: _____

Sat Sun Mon Tue Wed Thu Fri

Date: / /

* Right Outer Join (⟖):

L_no	C_name	b_name	Amount
L-16	Adams	PR	1300
L-11	Smiths	RH	900
L-93	Jones	NULL	NULL
L-14	Jackson	DT	1500

SQL: Select * from loan NATURAL RIGHT OUTER JOIN Borrower;

RA:

* Full Outer Join (⟗):

L_no	C_name	b_name	Amount
L-11	Smith	RH	900
L-14	Jackson	DT	1500
L-15	NULL	RR	1500
L-16	Adams	PR	1300
L-17	NULL	DT	1000
L-93	Jones	NULL	NULL

SQL:

RA:

CamScanner

Sub: _____

Sat Sun Mon Tue Wed Thu Fri

Date: / /

*Keys: Key is an attribute / Set of attributes for uniquely identifying each tuple/row.

4 types of Key:

1. Super Key
2. Candidate Key
3. Primary Key
4. Foreign Key

*Super key:

Student (Id, Email, Name, Dept)

{Id}	{Id, Dept}	{Id, Name, Email, Dept}
{Email}	{Email, Name}	
{Id, name}	{Email, Dept}	

*Candidate Key:

{Id} {Email}

Every candidate Key can be a Super key but each super key isn't a candidate key.

(i) A candidate key is a minimal super key; it uniquely identifies a row and has no unnecessary attribute.

CamScanner

Sub: _____

Sat Sun Mon Tue Wed Thu Fri

Date: / /

*Primary Key:

(i) The primary key is the candidate key selected to be the main identifier of rows.

(ii) It must be unique and cannot contain NULL values.

* Foreign Key:

(i) A foreign key is an attribute (or set of attributes) in one table that references a primary key or unique key in another table.

(ii) It helps establish relationships between tables and supports referential integrity.

Student (Id, Email, Name, Dept_Id) ⟶ Dept_Id = ~~primary~~ Foreign key

Department (Dept_Id, Dept_name, location) ⟶ Dept_Id = ~~Foreign~~ primary key

* Student (Id, Name, Email, dept_id)
* Department (Dept_id, Dept_name, location)

~~Create table Student (Id int(10). PRIMARY KEY, Name varchar(~~

CamScanner

Sub: _____ Sat Sun Mon Tue Wed Thu Fri

Date: / /

SQL for defining PK & FK:

⇒ Create table Student (Id int(10) PRIMARY KEY, Name varchar(30), Email varchar(30), dept_id varchar(10), FOREIGN KEY (dept_id) REFERENCES Department (dept_id));

⇒ Create table Department (Dept_id varchar(10) PRIMARY KEY, Dept_name varchar(10), location varchar(15));

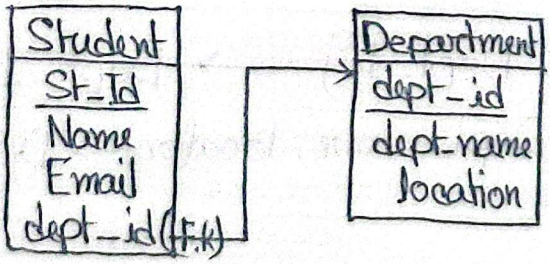
* Defining PK after creating a table:

Alter table Department ADD PRIMARY KEY (Dept_id);

* Schema Diagram:

Student (St_Id, Name, Email, dept_id) (dept_id: Fk)

Department (dept_id, dept_name, location)

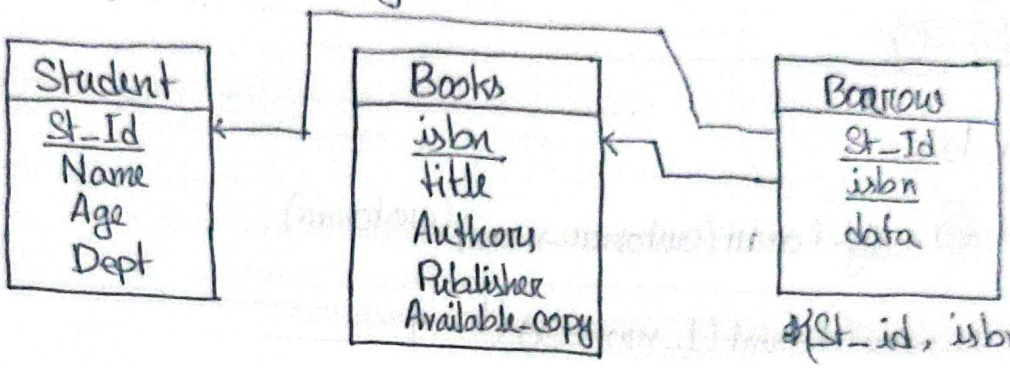


* Student (St_Id, Name, Age, Dept)
  Books (isbn, title, Author, Publisher$, Available_copy)
  Borrow (St_Id, isbn, data)

CamScanner

Sub: _____ Date: / /

Q. Find the suitable Pk & Fk for each table. Then draw the corresponding schema diagram.



{St_id, isbn} Composite Pk

~~* 3rd lab:~~

* 2nd Lab:

Q1. $\Pi_{branch_name}(loan)$

Q2. $\Pi_{loan_number}(\sigma_{branch_name = "Perryride"} \wedge amount > 300(loan))$

Q3. $\Pi_{loan_number}(\sigma_{branch_name = "Perryride"} \vee branch_name = "Downtown"(loan))$

Q5. $\Pi_{customer_name}(\sigma_{c_city \neq "Stamford"} \vee c_city \neq "Princeton" \vee c_city \neq "Harrison"(customer))$

Q7. $G_{count(Customer_name)}(Customer)$

$G_{count(Customer_name)}$ as Total no. of Customers (Customer)

Q6. $G_{max(balance)}$ as Maximum balance (Account)

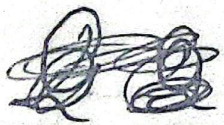
$G_{max(balance)}$ as Maximum, $min(balance)$ as Minimum, $avg(balance)$ as Average (Account)

Sub: _____

Sat Sun Mon Tue Wed Thu Fri

Date: / /

Q8. $\pi_{loan_number}(\sigma_{amount \geq 400 \wedge amount \leq 800}(loan))$



*3rd lab:

Q1. $_{c_city}\mathcal{G}_{count(customer_name)}(customer)$

Q2. $_{b_name}\mathcal{G}_{count(l_number)}(loan)$

CamScanner